Exercise 2

The art, ready to use

At the end, the pixel art imports crisp, one tile per world unit, the character sheet is cut into frames, and the project has its sorting layers.

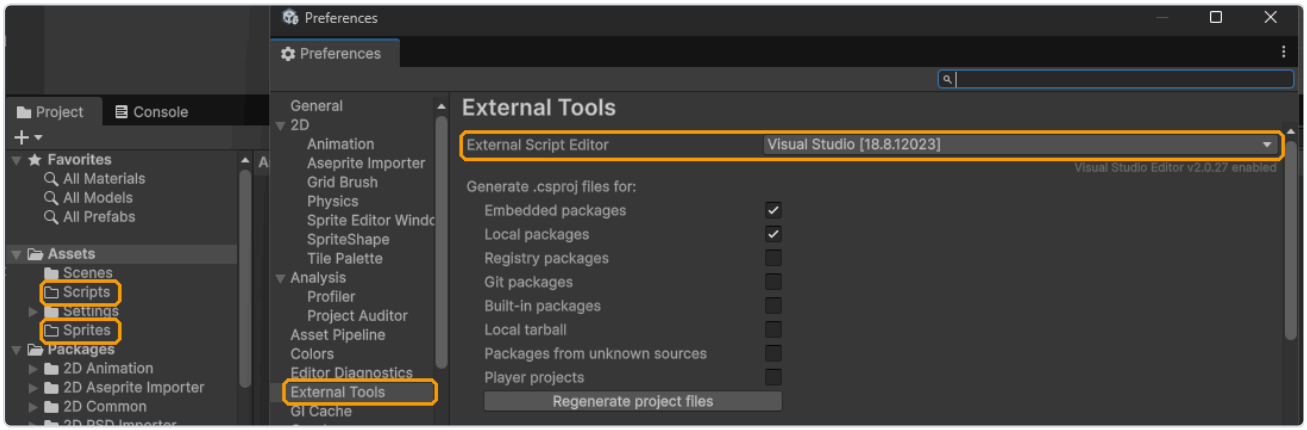
Before you start

- `PlatformerLab` on GitHub, from Exercise 1; in a terminal, in the project folder, run `git pull` first
- Kenney **Pixel Platformer** from kenney.nl/assets/pixel-platformer, unzipped outside the project (for example `D:\Unity\CourseAssets`)
- **Visual Studio 2026** with the **Game development with Unity** workload (macOS: Visual Studio Code with the Unity extension)

1 Prepare the project

Two folders keep the art and the code apart, and Unity needs to know which editor opens your scripts.

- In the Project window, right-click `Assets`, **Create** › **Folder**, and name it `Sprites`; make `Scripts` the same way (skip any that exist).
- **Edit** › **Preferences** › **External Tools**: set External Script Editor to the Visual Studio entry numbered 18, which is Visual Studio 2026, so a double-click on a script opens it there (macOS: Visual Studio Code).



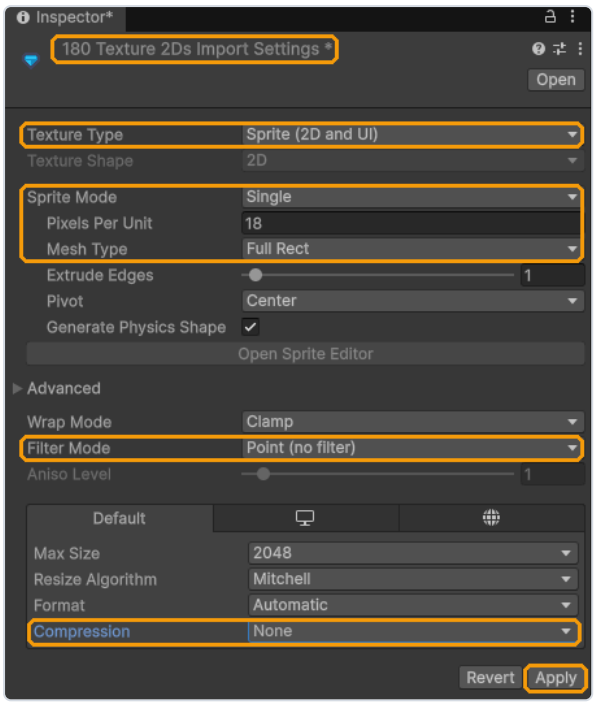
The two new folders under Assets, and Visual Studio [18.x] as the script editor.

2 Import the art

The pack arrives with Unity's defaults, made for large, smooth images; pixel art needs six settings changed, on all the files at once.

- From Explorer, drag the pack's `Tiles` folder onto `Assets/Sprites` in the Project window: you get 180 tiles plus the `Characters` and `Backgrounds` subfolders.
- Open `Assets/Sprites/Tiles`, click `tile_0000`, scroll down and Shift-click `tile_0179`: the Inspector header reads **180 Texture 2Ds Import Settings**.
- Set the six fields:
  - Texture Type: **Sprite (2D and UI)**, already set in a 2D project
  - Sprite Mode: **Single**, one sprite per file (the default here is Multiple)
  - Pixels Per Unit: `18`, the tile size, so one tile is one world unit
  - Mesh Type: **Full Rect**, which prevents hairline gaps between tiles
  - Filter Mode: **Point (no filter)**, which keeps the pixel edges sharp
  - Compression: **None**, which keeps the colors clean

- Press **Apply** and wait for the re-import.



The header counts the selected files; check the six fields, then Apply.

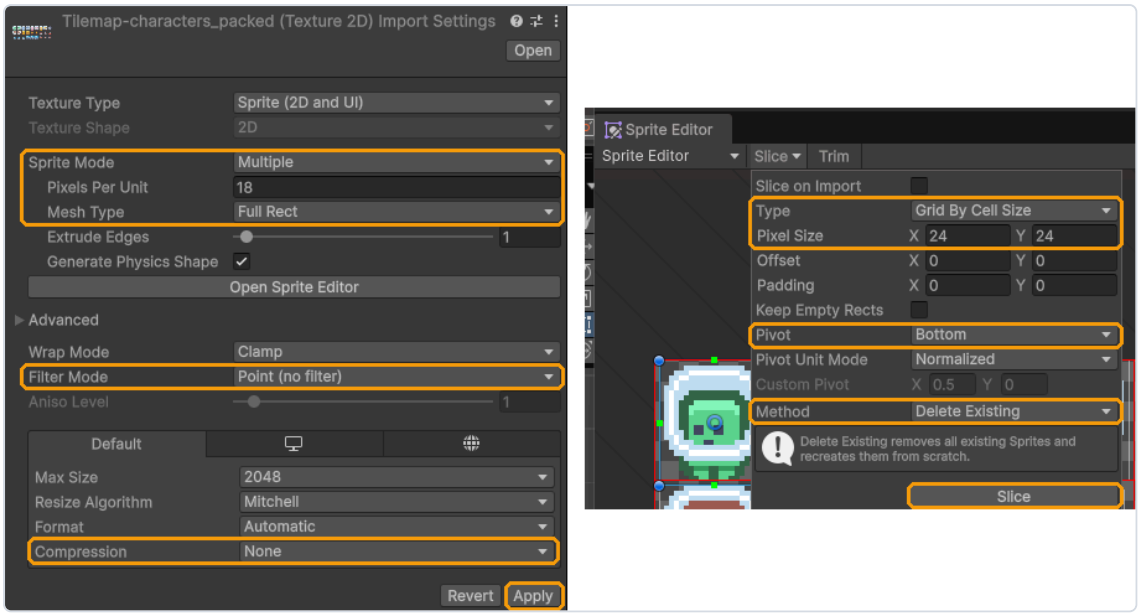
- The `Characters` and `Backgrounds` subfolders hold 24-pixel images that this project doesn't use: the characters come from one sheet, next.

3 The character sheet

The pack keeps every character in one image, a grid of 9 × 3 frames of 24 pixels. One import, cut by the Sprite Editor, gives the walker, and later the enemy, its frames.

- Drag `tilemap-characters_packed.png` from the pack's `Tilemap` folder into `Assets/Sprites` in the Project window.
- Select it and set **Sprite Mode** to `Multiple`, because the file holds many images, plus the project's usual values: Pixels Per Unit `18`, Mesh Type `Full Rect`, Filter Mode `Point (no filter)`, Compression `None`. **Apply**.

- Press **Open Sprite Editor**, open **Slice** and set Type `Grid By Cell Size`, Pixel Size `24` × `24`, Pivot `Bottom`, so a character's position is the point between its feet, and Method `Delete Existing`. Press **Slice**, then **Apply** in the Sprite Editor's toolbar, and close it.



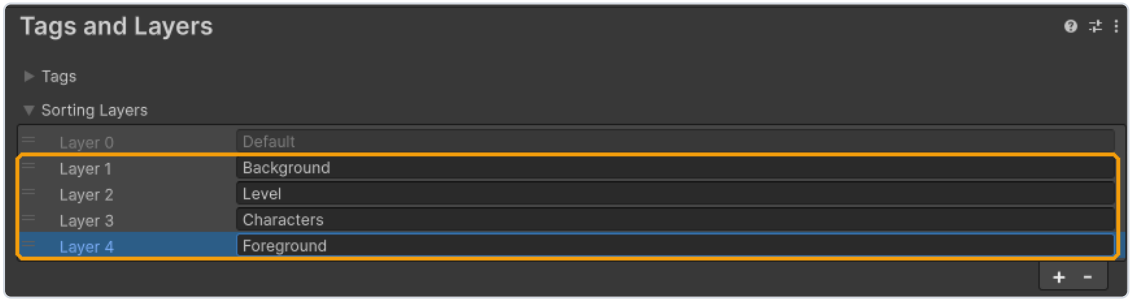
Multiple and the usual import values (left); a 24-pixel grid with bottom pivots (right).

- The sheet now has an arrow in the Project window: 27 sprites, `_0` to `_26`. `_0` is the astronaut standing, `_1` walking.

4 Sorting layers

Sorting layers decide what draws in front of what; the project gets four, once, for all its scenes.

- **Edit** › **Project Settings** › **Tags and Layers**, and expand **Sorting Layers**.
- Press **+** four times and name the layers `Background`, `Level`, `Characters`, `Foreground`, in this order below `Default`: a layer lower in the list draws in front.



Four new layers below Default, in this order.

- **File** › **Save Project**, then commit and push with the message `Import tiles, the character sheet and sorting layers`.